

A* Search Algorithm (A-star)

Learn How to Search Efficiently.

Made By DoDo

History

- Was co-developed by Peter Hart, Nils Nilsson and Bertram Raphael at the Stanford Research Institute.
- Was developed at **1968**.
- A* is very optimal because it uses both the distance already traveled and an estimate of the distance left.



How Does An A* Search Work:

Step 1

Start Node

A* begins at the starting node.

Step 2

Calculate Costs

It calculates $f(n)=g(n)+h(n)$.

Step 3

Pick Best Node

It chooses the node with the lowest f-value.

Step 4

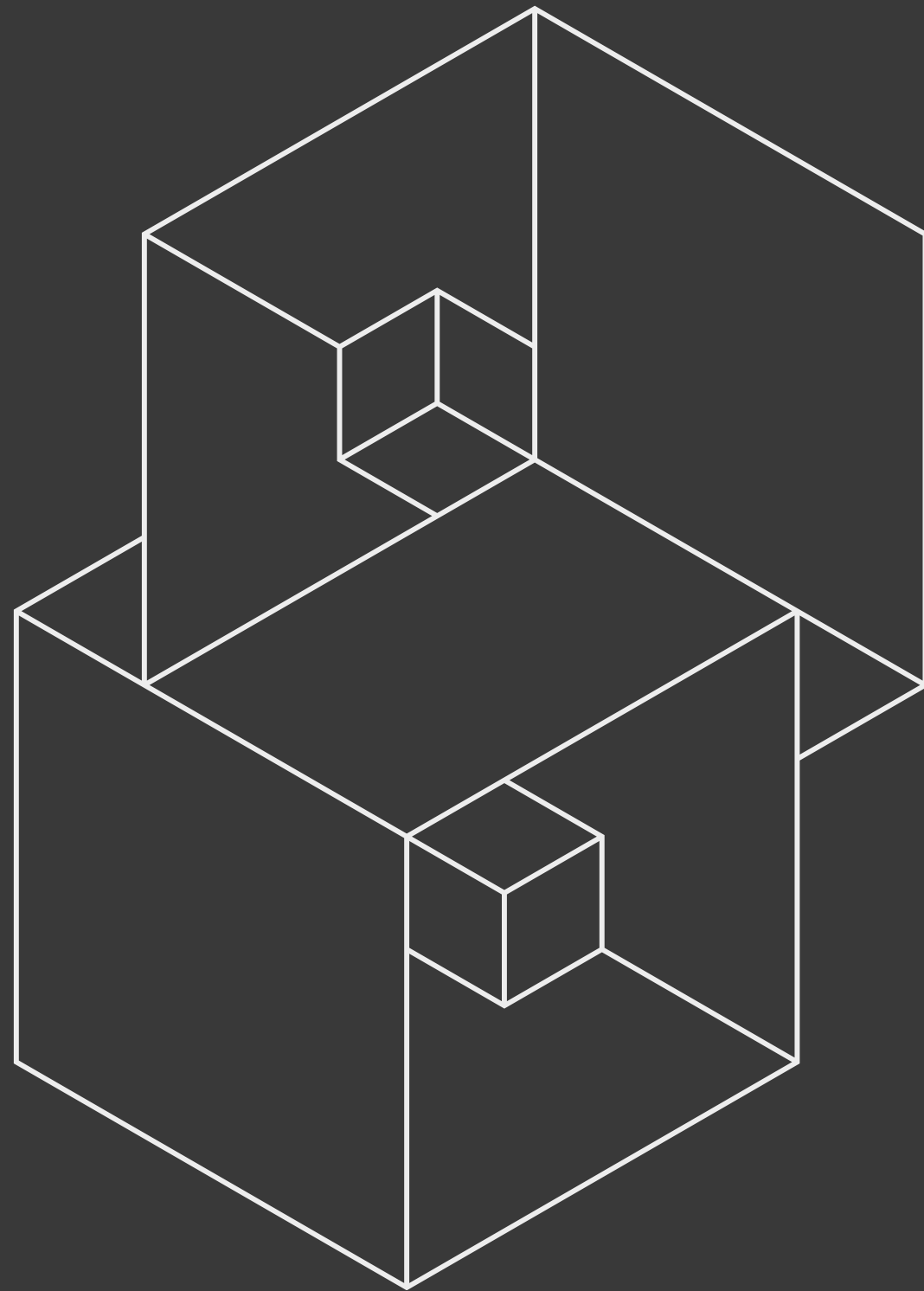
Explore Neighbors

It checks nearby nodes and updates their costs.

Step 4

Reach the Goal

It continues until the goal is found.



What Does This Mean?

$$f(n) = g(n) + h(n)$$

- $g(n)$: Cost from the start to node n
- $h(n)$: Estimated cost from node n to the goal
- $f(n)$: Total estimated path cost

Meaning: Actual Cost + Estimated Remaining Cost.

Cons and The Pros of The A* Search:

Pros:

VS

Cons:

- Finds the shortest path.
- Faster than most other search algorithms .
- Will only focus on the target.

- Uses a lot of memory.
- Slower on very large or complex graphs.
- Not optimal when there are multiple targets.

Example:

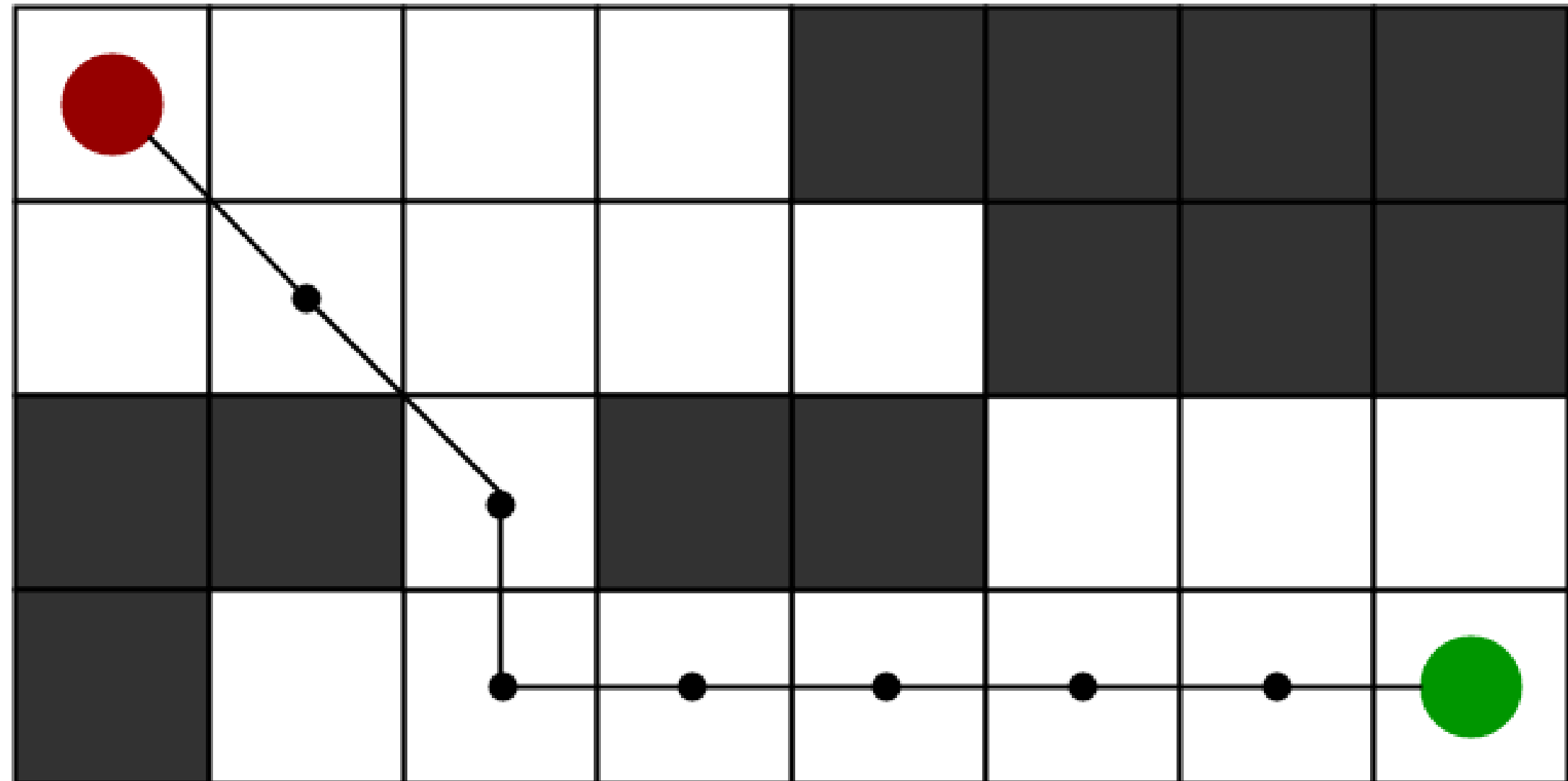
In this example A* starts at the **red** aiming for **green**. For each move it uses the formula

$$f(n) = g(n) + h(n)$$

-g(n) = Cost so far

-h(n) = Estimated cost goal.

By comparing these values A* avoids the blocked squares and picks the path with the lowest total cost.



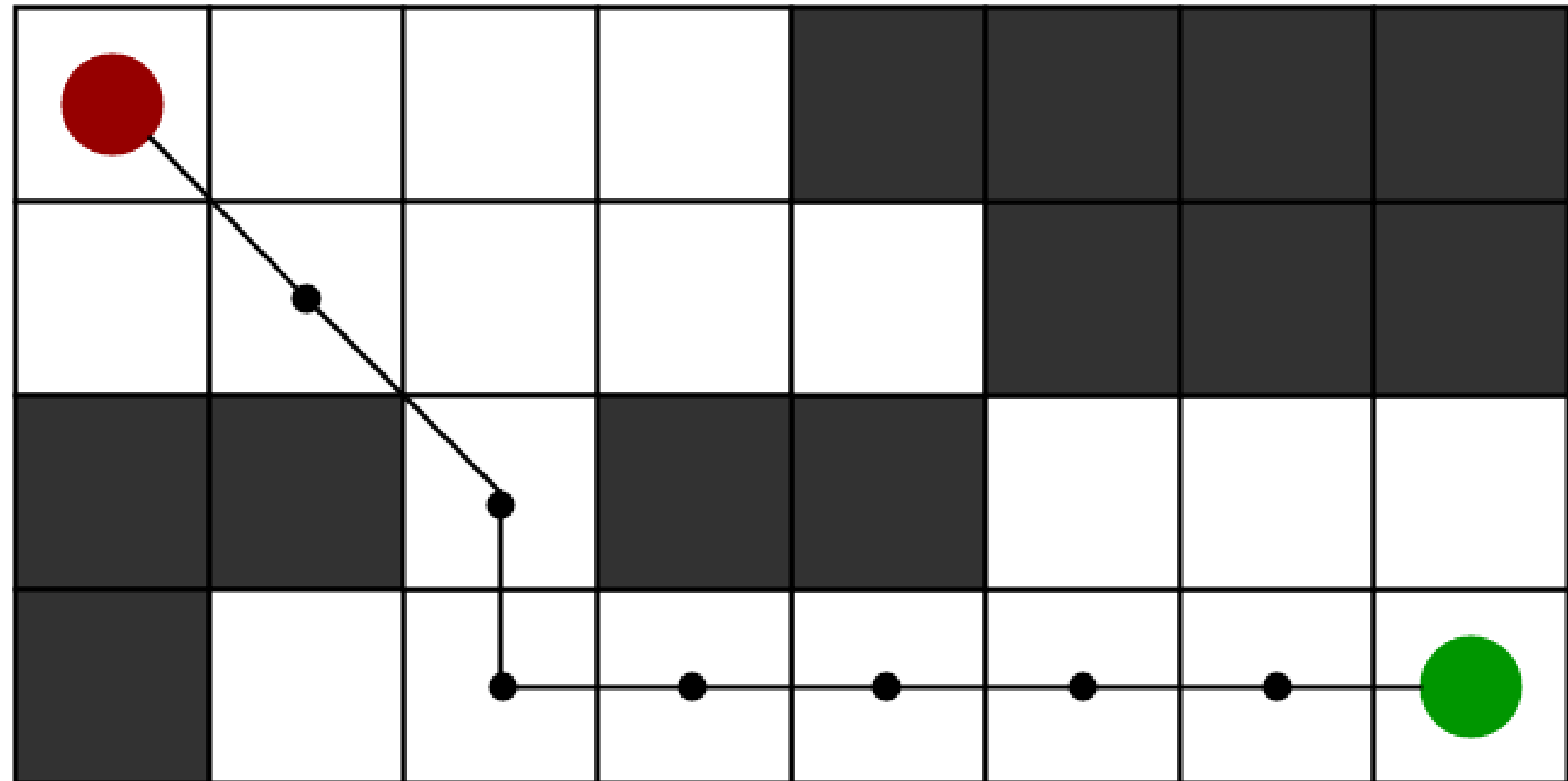
Example:

In step 1 the algorithm starts at the **red** and checks the two neighboring squares. If each move costs 1 then for both of these squares $g(n)=1$.

If we use Manhattan distance (which means counting how many squares are left to move horizontally and vertically) then $h(n)=9$.

$f(n)=1+9=10$.

A* compares the values and chooses the square with the lowest total cost.



Pseudocode Example:

```
GRID = 5x5 array (0=open, 1=wall)           // the map
GOAL = (4, 4)                                // where we want to go

open = priority queue sorted by f             // cells to check, best first
add start (0,0) to open                       // begin at top-left

while open is not empty:                     // keep going until done
    current = pop lowest f from open           // pick the most promising cell

    if current == GOAL:                       // did we reach the end?
        trace path back to start              // follow parents back
        print path                            // show the result
        stop                                  // we're done

    for each neighbor (up,down,left,right): // check all 4 directions
        if out of bounds or wall:             // skip invalid cells
            skip

        g = steps so far + 1                  // cost to reach neighbor
        h = manhattan distance to goal        // estimated cost to goal
        f = g + h                             // total score

        add neighbor to open with f           // queue it up to explore
        remember where we came from           // so we can trace the path back

print "No path found"                         ↓ / open emptied, no route exists
```


Q1) Find $g(n)$ Value at Step 1:

Grid — red = start | green = goal | dark = wall | dots = path



A) 0

B) 1

C) 4

D) 8

Q1) Find $g(n)$ Value at Step 1:

Grid — red = start | green = goal | dark = wall | dots = path



B) 1

G is the steps taken from the start.

After the first move it should be $G = 1$.

Q1) Find $h(n)$ Value at Step 1:

Grid — red = start | green = goal | dark = wall | dots = path



A) 4

B) 6

C) 8

D) 10

Q1) Find $h(n)$ Value at Step 1:

Grid — red = start | green = goal | dark = wall | dots = path



D) 10

To calculate $h(n)$ we need to add the row and the col difference which is: $4 + 6 = 10$.